



Dr. Alessandro Agostini

Homework 3

Issued Sunday 24 November. Due as **zipped archive (.tex, .pdf, images) to eClass by Monday 2 December 2024 at noon (12:00, system's clock)**. Hardcopy: none (upon request for specific cases).

Please note the following:

0. Deadline is **strict**. **Electronic deadline passed without submission implies mark 0 (zero).** (Equivalently, you will not be admitted to enter the classroom for Quiz 3.)

1. The suggested number of students is 1 to 3. We allow teams formed by maximum 4 members.

1.1 Teams formed by more than 4 members **are not permitted** (students not admitted to Quiz 3).

2. Although teams can be formed by students attending any of the sections 001 to 004, it is preferable all students are in the same section. In any case, students are required to write their **section number** — example: 001; note: not the group (CSE... or ICE...) — in the cover of submitted work (see 3 below).

IMPORTANT: Only students listed in the cover of submitted work are admitted to Quiz 3. In other words, students whose ID, full name and section number do not appear on the cover **are not admitted to enter the classroom for Quiz 3 and are assigned score 0/100 for the homework**.

IMPORTANT: Submissions **without cover** and **missing the .pdf file** are rejected without review.

IMPORTANT: Students in teams who got cheating in Homework #1 or #2 can't submit this homework.

3. L^AT_EX's template and Submission. The homework must be set by using L^AT_EX's template provided (see (*) below). You are asked to submit a **zipped archive** containing L^AT_EX's file (.tex), its compilation (.pdf) (using command *pdflatex*), and a folder with all the images included in the .tex file. All what required is subject of evaluation and points are assigned as a consequence of quality of submission.

(*) The main document (pdf file) you submit must be produced from the source file provided (cf. L^AT_EX's template in eClass, Week 12) by using L^AT_EX, specifically one of following versions:
1. <https://ctan.org/pkg/texlive> (general) or 2. <https://ctan.org/pkg/mactex> (MacOS users).^a

^aIf you have serious issues in installing any of such software packages, you are allowed to use OVERLEAF (<https://www.overleaf.com>), provided that the source file .tex produced is compatible with TexLive.

4. Only one member of the team submits the document.

IMPORTANT: Multiple submission by members of the same team **are penalized** as follows: For each team member who submits the document already submitted, scoring (of related Quiz 2) is reduced by 5/100 points for all team members. **Example:** A team of three, three members submit: score of Quiz 2 reduced by 10 points (over 100 points) for all team members.

4.1 The **name of the submitted zipped archive** as well the **name of each submitted file**: .tex file and .pdf file, is required to be in the following format: 00X-ID (example: 001-U1910123), where:

- 00X is the section of the team's member, say S, who submits the homework to eCLASS;
- ID is the ID number of S.

5. REMINDER (from Syllabus): Cheating in homework assignments implies 0/100 for all members of the group on **Participation and all Homework Assignments** (so: 0/100 for 35% of final score).

PARTICULAR SETTING:

None.

PROBLEMS AND EXERCISES:

TECHNICAL NOTE: (1) In exercises where you need to translate a SQL query into RA (Relational Algebra) or ERA (Extended or Generalized RA), if the SQL query contains clauses `IN` or `NOT IN`, use symbols $\in, \notin$, respectively, in your translation. If the SQL query contains clauses `EXISTS` or `NOT EXISTS` before a certain subquery, say Q , then write “ $\exists x Q(x)$ ”, “ $\neg \exists x Q(x)$ ”, respectively, in your translation. If the SQL query contains clauses `ALL` or `SOME`, with any arithmetic symbol for comparison $<, <=, =, <>, >=, >$ on the left, before a certain subquery, say S , then translate it by writing, for example, “ $\forall x \in S (A < s)$ ”, if `<ALL` is used to compare attribute A ’s value with all members of S . (Similarly, write “ $\exists x \in S (A < s)$ ”, if `<SOME` is used.) (2) In the exercises where you need to know the logical schema of a relation, refer to the “University DB” schema showed in Figure 2 (cf. last page). (3) Remember: for the sake of translation from SQL into RA, ERA (GRA), TRC, or DRC, SQL’s `SELECT` and `SELECT DISTINCT`, and `COUNT (DISTINCT ...)` and `COUNT (...)`, are equivalent.

Definition 1 Two queries on a relational database **DB** are *language-equivalent* if (a) they are equivalent, that is, they produce the same structure (schema) and content (instance) over **DB**’ schema and instance, respectively, and (b) the translation from one query to the other (and vice versa) is purely algorithmic, i.e., expressivity of language in input is preserved in output.

Terminology: RA, ERA (GRA), TRC, DRC are acronyms of “Relational Algebra”, “Extended (or: Generalized) Relational Algebra”, “Tuple Relational Calculus”, and “Domain Relational Calculus”, respectively.

Convention: To shorten DRC’s expressions, we used only the initial letter or (if ambiguous, the first two-three letters) of attributes’ name used in each relation schema. We used full name of relations or, if renamed (by using `AS` clause in SQL), the relation’s name as renamed (“alias”).

1 Translate from English

Exercise 1 [Fall 2024, Week 11 (“Bonus Week”); *revised*] Consider the following English query:

Find names of all students who possibly take a course and whose total credits (tot_cred) are less or equal than the total credits of each student in Computer Science.

1. Write a *language-equivalent* query in SQL (cf. Definition 1, page 2).
2. Write a *language-equivalent* RA’s expression (cf. Definition 1, page 2).
3. (optional) Write a *language-equivalent* TRC’s expression (cf. Definition 1, page 2).
4. (optional) Write a *language-equivalent* DRC’s expression (cf. Definition 1, page 2).

Exercise 2 [Fall 2024, Week 11 (“Bonus Week”); *revised*] Consider the following English query:

Find ID, name of students, and their grade A+, for students who got 2 or more A+ in taking a course.

1. Write a *language-equivalent* query in SQL (cf. Definition 1, page 2).
2. Write a *language-equivalent* RA's expression (cf. Definition 1, page 2).

Exercise 3 [Fall 2024, Week 11 (“Bonus Week”); *revised*] Consider the following English query:

Find departments' name along with number of instructors in the department and, for each department, also find the average salary for those departments whose average salary is greater than 45000 (dollars).

1. Write a *language-equivalent* query in SQL (cf. Definition 1, page 2).
2. Write a *language-equivalent* RA's expression (cf. Definition 1, page 2).

2 Translate from SQL

Remark 1 For each SQL query in exercises below, if the query is syntactically correct and executes in MySQL, then do what asked. Otherwise, explain the error, rewrite the query in SQL standard that executes in MySQL, and do what asked starting from the corrected query.

Exercise 4 [Fall 2024, Week 11 (“Bonus Week”); *revised*] Consider the following SQL query:

```
SELECT DISTINCT I.salary
FROM instructor AS I, instructor AS B
WHERE I.salary < B.salary AND B.dept_name = 'Biology';
```

1. Write a *language-equivalent* RA's expression (cf. Definition 1, page 2).
2. Write a *language-equivalent* TRC's expression (cf. Definition 1, page 2).
3. Write a *language-equivalent* DRC's expression (cf. Definition 1, page 2).

Exercise 5 [Fall 2024, Week 11 (“Bonus Week”); *revised*] Consider the following SQL query:

```
SELECT name, course_id
FROM (SELECT * FROM takes
      WHERE year >= 2008 ) NATURAL JOIN student;
```

1. Write a *language-equivalent* RA's expression (cf. Definition 1, page 2).
2. Write a *language-equivalent* TRC's expression (cf. Definition 1, page 2).
3. Write a *language-equivalent* DRC's expression (cf. Definition 1, page 2).

Exercise 6 [Fall 2024, Week 11 (“Bonus Week”); *revised*] Consider the following SQL query:

```
SELECT dept_name, AVG (salary) AS avg_salary
FROM instructor
GROUP BY dept_name
HAVING avg_salary > 73000;
```

1. Write a *language-equivalent* RA's expression (cf. Definition 1, page 2).

Exercise 7 [Fall 2024, Week 11 ("Bonus Week"); *revised*] Consider the following SQL query:

```
(SELECT course_id FROM section
WHERE semester = Fall AND year = 2009)
UNION
(SELECT course_id FROM section
WHERE semester = Spring AND year = 2010);
```

1. Write a *language-equivalent* RA's expression (cf. Definition 1, page 2).
2. Write a *language-equivalent* TRC's expression (cf. Definition 1, page 2).
3. Write a *language-equivalent* DRC's expression (cf. Definition 1, page 2).

Exercise 8 [Fall 2024, Week 11 ("Bonus Week"); *revised*] Consider the following SQL query:

```
(SELECT course_id FROM section
WHERE semester = Fall AND year = 2009)
INTERSECT
(SELECT course_id FROM section
WHERE semester = Spring AND year = 2010);
```

1. Write a *language-equivalent* RA's expression (cf. Definition 1, page 2).
2. Write a *language-equivalent* TRC's expression (cf. Definition 1, page 2).
3. Write a *language-equivalent* DRC's expression (cf. Definition 1, page 2).

Exercise 9 [Fall 2024, Week 11 ("Bonus Week"); *revised*] Consider the following SQL query:

```
(SELECT course_id FROM section
WHERE semester = Fall AND year = 2009)
EXCEPT
(SELECT course_id FROM section
WHERE semester = Spring AND year = 2010);
```

1. Write a *language-equivalent* RA's expression (cf. Definition 1, page 2).
2. Write a *language-equivalent* TRC's expression (cf. Definition 1, page 2).
3. Write a *language-equivalent* DRC's expression (cf. Definition 1, page 2).

Exercise 10 [Fall 2024, Week 11 ("Bonus Week"); *revised*] Consider the following SQL query:

```
SELECT best.ID, instructor.salary
FROM (SELECT MAX(test.cnt), ID
      FROM (SELECT ID, COUNT(DISTINCT course_id) AS cnt
            FROM teaches
            GROUP BY ID) AS test)
      GROUP BY ID) AS best, instructor
WHERE instructor.ID = best.ID;
```

1. Write a *language-equivalent* RA's expression (cf. Definition 1, page 2).

Exercise 11 [Fall 2024, Week 11 (“Bonus Week”); *revised*] Consider the following SQL query:

```
SELECT name FROM instructor AS I
WHERE dept_name = 'Comp.Sci' OR salary > 70000;
```

1. Write a *language-equivalent* RA's expression (cf. Definition 1, page 2).
2. Write a *language-equivalent* TRC's expression (cf. Definition 1, page 2).
3. Write a *language-equivalent* DRC's expression (cf. Definition 1, page 2).

Exercise 12 [Fall 2024, Week 11 (“Bonus Week”); *revised*] Consider the following SQL query:

```
SELECT name, D.dept_name, building
FROM instructor AS I CROSS JOIN department AS D
WHERE I.dept_name = D.dept_name;
```

1. Write a *language-equivalent* RA's expression (cf. Definition 1, page 2).
2. Write a *language-equivalent* TRC's expression (cf. Definition 1, page 2).
3. Write a *language-equivalent* DRC's expression (cf. Definition 1, page 2).

Exercise 13 [Fall 2024, Week 11 (“Bonus Week”); *revised*] Consider the following SQL query:

```
/* modified from Week 11 */
SELECT *
FROM instructor AS I JOIN department AS D ON I.dept_name = D.dept_name;
```

1. Write a *language-equivalent* RA's expression (cf. Definition 1, page 2).
2. Write a *language-equivalent* TRC's expression (cf. Definition 1, page 2).
3. Write a *language-equivalent* DRC's expression (cf. Definition 1, page 2).

Exercise 14 [Fall 2024, Week 11 (“Bonus Week”); *revised*] Consider the following SQL query:

```
SELECT * FROM instructor AS I NATURAL JOIN department;
```

1. Write a *language-equivalent* RA's expression (cf. Definition 1, page 2).
2. Write a *language-equivalent* TRC's expression (cf. Definition 1, page 2).
3. Write a *language-equivalent* DRC's expression (cf. Definition 1, page 2).

Exercise 15 [Fall 2024, Week 11 (“Bonus Week”); *revised*] Consider the following SQL query:

```

/* modified from Week 11 */
SELECT name, course_id
FROM instructor CROSS JOIN teaches
WHERE instructor.ID = teaches.ID;

```

1. Write a *language-equivalent* RA's expression (cf. Definition 1, page 2).
2. Write a *language-equivalent* TRC's expression (cf. Definition 1, page 2).
3. Write a *language-equivalent* DRC's expression (cf. Definition 1, page 2).

Exercise 16 [Fall 2024, Week 11 (“Bonus Week”); *revised*] Consider the following SQL query:

```

SELECT name, title FROM instructor
NATURAL JOIN teaches NATURAL JOIN course;

```

1. Write a *language-equivalent* RA's expression (cf. Definition 1, page 2).
2. Write a *language-equivalent* TRC's expression (cf. Definition 1, page 2).
3. Write a *language-equivalent* DRC's expression (cf. Definition 1, page 2).

Exercise 17 [Fall 2024, Week 11 (“Bonus Week”); *revised*] Consider the following SQL query:

```

SELECT COUNT (*) FROM accident
WHERE EXISTS (SELECT *
              FROM participated AS P, owns, person
              WHERE owns.P_driver_id = person.driver_id
                AND person.name = 'John Smith' AND owns.C_plate_num =
                P.C_plate_num
                AND accident.report_num = P.A_report_num);

```

on the logical schema as designed here:

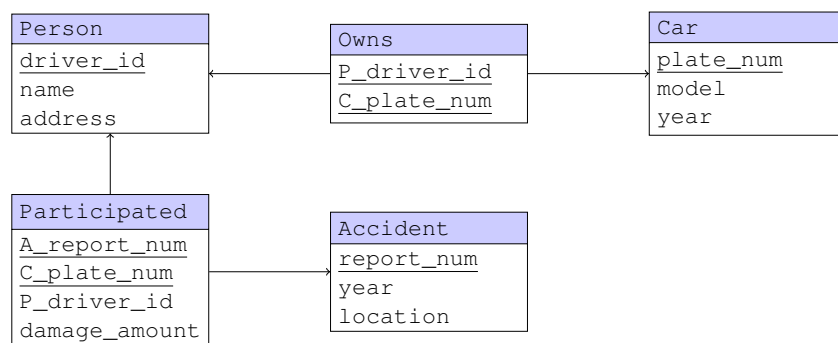


Figure 1: Logical schema of Insurance Database (Silberschatz *et al.*, 2011/2020; *modified*).

1. Write a *language-equivalent* RA's expression (cf. Definition 1, page 2).
2. Write a *language-equivalent* TRC's expression (cf. Definition 1, page 2).

3. Write a *language-equivalent* DRC's expression (cf. Definition 1, page 2).

Exercise 18 [Fall 2024, Week 11 (“Bonus Week”); *revised*] Consider the following SQL query:

```
SELECT ID, name
FROM instructor
WHERE ID NOT IN (SELECT ID
                  FROM instructor NATURAL JOIN teaches);
```

1. Write a *language-equivalent* RA's expression (cf. Definition 1, page 2).
2. Write a *language-equivalent* TRC's expression (cf. Definition 1, page 2).
3. Write a *language-equivalent* DRC's expression (cf. Definition 1, page 2).

Exercise 19 [Fall 2024, Week 11 (“Bonus Week”); *revised*] Consider the following SQL query:

```
SELECT course_id, title, dept_name
FROM takes NATURAL JOIN course
WHERE ID = '12345';
```

1. Write a *language-equivalent* RA's expression (cf. Definition 1, page 2).
2. Write a *language-equivalent* TRC's expression (cf. Definition 1, page 2).
3. Write a *language-equivalent* DRC's expression (cf. Definition 1, page 2).

Exercise 20 [Fall 2024, Week 11 (“Bonus Week”); *revised*] Consider the following SQL query:

```
/* modified from Week 11 */
SELECT course_id, sec_id, semester, year, SUM(tot_cred)
FROM takes NATURAL JOIN student
WHERE year = 2021
GROUP BY course_id, sec_id, semester, year;
```

1. Write a *language-equivalent* RA's expression (cf. Definition 1, page 2).

Exercise 21 [Fall 2024, Week 11 (“Bonus Week”); *revised*] Consider the following SQL query:

```
/* modified from Week 11 */
SELECT (course_id, semester, year, sec_id) AS course-section,
       SUM(tot_cred) AS sum_tot_cred
FROM takes NATURAL JOIN student
WHERE year = 2021
GROUP BY (course_id, semester, year, sec_id);
HAVING COUNT(ID) <= 9;
```

1. Write a *language-equivalent* RA's expression (cf. Definition 1, page 2).

Exercise 22 [Fall 2024, Week 11 (“Bonus Week”); *revised*] Consider the following SQL query:

```
/* modified from Week 11 */  
SELECT *  
FROM student CROSS JOIN takes  
WHERE name NOT IN ('Abelard', 'Eloise');
```

1. Write a *language-equivalent* RA’s expression (cf. Definition 1, page 2).
2. Write a *language-equivalent* TRC’s expression (cf. Definition 1, page 2).
3. Write a *language-equivalent* DRC’s expression (cf. Definition 1, page 2).

Exercise 23 [Fall 2024, Week 11 (“Bonus Week”); *revised*] Consider the following SQL query:

```
/* modified from Week 11 */  
SELECT *  
FROM student NATURAL JOIN takes  
WHERE name NOT IN ('Abelard', 'Eloise');
```

1. Write a *language-equivalent* RA’s expression (cf. Definition 1, page 2).
2. Write a *language-equivalent* TRC’s expression (cf. Definition 1, page 2).
3. Write a *language-equivalent* DRC’s expression (cf. Definition 1, page 2).

Exercise 24 [Fall 2024, Week 11 (“Bonus Week”); *revised*] Consider the following SQL query:

```
SELECT *  
FROM student AS S JOIN takes AS T ON S.ID = T.ID  
WHERE name NOT IN ('Abelard', 'Eloise');
```

1. Write a *language-equivalent* RA’s expression (cf. Definition 1, page 2).
2. Write a *language-equivalent* TRC’s expression (cf. Definition 1, page 2).
3. Write a *language-equivalent* DRC’s expression (cf. Definition 1, page 2).

Exercise 25 [Fall 2024, Week 11 (“Bonus Week”); *revised*] Consider the following SQL query:

```
SELECT *  
FROM student AS S NATURAL JOIN takes AS T  
WHERE name NOT IN ('Abelard', 'Eloise');
```

1. Write a *language-equivalent* RA’s expression (cf. Definition 1, page 2).
2. Write a *language-equivalent* TRC’s expression (cf. Definition 1, page 2).
3. Write a *language-equivalent* DRC’s expression (cf. Definition 1, page 2).

Exercise 26 [Fall 2024, Week 11 (“Bonus Week”); *revised*] Consider the following SQL query:

```
SELECT name, course_id
FROM ( SELECT * FROM takes
      WHERE year >= 2008 ) NATURAL JOIN student;
```

1. Write a *language-equivalent* RA’s expression (cf. Definition 1, page 2).
2. Write a *language-equivalent* TRC’s expression (cf. Definition 1, page 2).
3. Write a *language-equivalent* DRC’s expression (cf. Definition 1, page 2).

Exercise 27 [Fall 2024, Week 11 (“Bonus Week”); *revised*] Consider the following SQL query:

```
SELECT name, course_id
FROM takes JOIN student USING (ID)
WHERE EXISTS (SELECT * FROM takes
             WHERE year >= 2008 );
```

1. Write a *language-equivalent* RA’s expression (cf. Definition 1, page 2).
2. Write a *language-equivalent* TRC’s expression (cf. Definition 1, page 2).
3. Write a *language-equivalent* DRC’s expression (cf. Definition 1, page 2).

Exercise 28 [Fall 2024, Week 11 (“Bonus Week”); *revised*] Consider the following SQL query:

```
SELECT dept_name, AVG (salary) AS avg_salary
FROM instructor
GROUP BY dept_name
HAVING avg_salary > 73000;
```

1. Write a *language-equivalent* RA’s expression (cf. Definition 1, page 2).

Exercise 29 [Fall 2024, Week 11 (“Bonus Week”); *revised*] Consider the following SQL query:

```
SELECT dept_name, avg_salary
FROM (SELECT dept_name, AVG(salary) AS avg_salary
      FROM instructor GROUP BY dept_name)
WHERE EXISTS (SELECT dept_name, AVG(salary) AS tmp_avg_salary
             FROM instructor
             GROUP BY dept_name
             HAVING tmp_avg_salary=avg_salary
                  AND tmp_avg_salary > 73000);
```

1. Write a *language-equivalent* RA’s expression (cf. Definition 1, page 2).

Exercise 30 [Fall 2024, Week 11 (“Bonus Week”); *revised*] Consider the following SQL query:

```

/* modified from Week 11 */
(SELECT title
 FROM course
 WHERE dept_name in ('ComputerScience', 'History')
 EXCEPT
 (SELECT title
  FROM student NATURAL JOIN takes
        JOIN course USING (course_id)
  WHERE tot_cred > SOME (SELECT tot_cred
                        FROM student
                        WHERE dept_name = 'ComputerScience'));

```

1. Write a *language-equivalent* RA's expression (cf. Definition 1, page 2).
2. Write a *language-equivalent* TRC's expression (cf. Definition 1, page 2).
3. Write a *language-equivalent* DRC's expression (cf. Definition 1, page 2).

Exercise 31 [Fall 2024, Week 11 ("Bonus Week"); *revised*] Consider the following SQL query:

```

SELECT ID, dept_name
FROM takes NATURAL JOIN course
WHERE ID = '12345' OR dept_name = 'CSE';

```

1. Write a *language-equivalent* RA's expression (cf. Definition 1, page 2).
2. Write a *language-equivalent* TRC's expression (cf. Definition 1, page 2).
3. Write a *language-equivalent* DRC's expression (cf. Definition 1, page 2).

Exercise 32 [Fall 2024, Week 11 ("Bonus Week"); *revised*] Consider the following SQL query:

```

SELECT course_id, title
FROM student, takes NATURAL JOIN course
WHERE course_id = '3020' AND title = 'Database';

```

1. Write a *language-equivalent* RA's expression (cf. Definition 1, page 2).
2. Write a *language-equivalent* TRC's expression (cf. Definition 1, page 2).
3. Write a *language-equivalent* DRC's expression (cf. Definition 1, page 2).

Exercise 33 [Fall 2024, Week 11 ("Bonus Week"); *revised*] Consider the following SQL query:

```

SELECT course_id, title
FROM instructor NATURAL JOIN teaches, course
WHERE course_id NOT IN ('3020', '2310') OR title = 'Database';

```

1. Write a *language-equivalent* RA's expression (cf. Definition 1, page 2).
2. Write a *language-equivalent* TRC's expression (cf. Definition 1, page 2).

3. Write a *language-equivalent* DRC's expression (cf. Definition 1, page 2).

Exercise 34 [Fall 2024, Week 11 (“Bonus Week”); *revised*] Consider the following SQL query:

```
/* modified from Week 11 */
SELECT *
FROM instructor NATURAL JOIN teaches AS T
JOIN course AS C ON T.course_id = C.course_id
WHERE C.course_id NOT IN ('3020', '2310') OR title = 'Database';
```

1. Write a *language-equivalent* RA's expression (cf. Definition 1, page 2).
2. Write a *language-equivalent* TRC's expression (cf. Definition 1, page 2).
3. Write a *language-equivalent* DRC's expression (cf. Definition 1, page 2).

Exercise 35 [Fall 2024, Week 11 (“Bonus Week”); *revised*] Consider the following SQL query:

```
SELECT SUM(credits) AS sumC, ID
FROM takes AS T NATURAL JOIN course
GROUP BY T.ID;
```

1. Write a *language-equivalent* RA's expression (cf. Definition 1, page 2).

3 Translate from RA or ERA

For each RA's expression in exercises below, if the expression is syntactically correct, then do what asked. Otherwise, explain the error, rewrite the expression correctly and do what asked starting from the corrected expression.

Exercise 36 [Fall 2024, Week 11 (“Bonus Week”); *revised*] Consider the following RA's expression:

$$\Pi_{name}(\sigma_{dept_name='Comp.Sci' \wedge salary > 70000}(instructor))$$

1. Write a *language-equivalent* query in SQL standard (cf. Definition 1, page 2).

Exercise 37 [Fall 2024, Week 11 (“Bonus Week”); *revised*] Consider following RA's expression(s):

$$\begin{aligned} s &\leftarrow instructor \times department \\ t &\leftarrow \sigma_{instructor.dept_name=department.dept_name}(s) \\ &\Pi_{name, dept_name, building}(t) \end{aligned}$$

1. Write a *language-equivalent* query in SQL standard (cf. Definition 1, page 2).

Exercise 38 [Fall 2024, Week 11 (“Bonus Week”); *revised*] Consider following RA's expression(s):

$$\begin{aligned} s &\leftarrow instructor \times teaches \\ &\Pi_{name, course_id}(\sigma_{instructor.ID=teaches.ID}(s)) \end{aligned}$$

1. Write a *language-equivalent* query in SQL standard (cf. Definition 1, page 2).

Exercise 39 [Fall 2024, Week 11 (“Bonus Week”); *revised*] Consider following RA’s expression(s):

$$s \leftarrow (instructor + teaches) \times course$$

$$\Pi_{name,title}(\sigma_{teaches.course_id=course.course_id}(s))$$

1. Write a *language-equivalent* query in SQL standard (cf. Definition 1, page 2).

Exercise 40 [Fall 2024, Week 11 (“Bonus Week”); *revised*] Consider following RA’s expression(s):

$$s \leftarrow (instructor + \rho_T(teaches)) \times \rho_C(course)$$

$$\Pi_{name,title}(\sigma_{T.course_id=C.course_id}(s))$$

1. Write a *language-equivalent* query in SQL standard (cf. Definition 1, page 2).

Exercise 41 [Fall 2024, Week 11 (“Bonus Week”); *revised*] Consider following RA’s expression(s):

$$s \leftarrow instructor + (teaches \times course)$$

$$\Pi_{name,title}(\sigma_{teaches.course_id=course.course_id}(s))$$

1. Write a *language-equivalent* query in SQL standard (cf. Definition 1, page 2).

Exercise 42 [Fall 2024, Week 11 (“Bonus Week”); *revised*] Consider following RA’s expression(s):

$$s \leftarrow \rho_T(takes) + course$$

$$\Pi_{course_id,title}(\sigma_{ID='12345'}(s))$$

1. Write a *language-equivalent* query in SQL standard (cf. Definition 1, page 2).

Exercise 43 [Fall 2024, Week 11 (“Bonus Week”); *revised*] Consider following RA’s expression(s):

$$s \leftarrow \rho_{T(ID, course_id, sec_id, semester, year, grade)}(takes)$$

$$t \leftarrow s \bowtie course$$

$$\Pi_{course_id,title}(\sigma_{ID='12345'}(t))$$

1. Write a *language-equivalent* query in SQL standard (cf. Definition 1, page 2).

Exercise 44 [Fall 2024, Week 11 (“Bonus Week”); *revised*] Consider following RA’s expression(s):

$$s \leftarrow \rho_{T(ID, course_id, sec_id, semester, year, grade)}(takes)$$

$$t \leftarrow s \bowtie course$$

$$\sigma_{ID='12345'}(\Pi_{ID,title}(t))$$

1. Write a *language-equivalent* query in SQL standard (cf. Definition 1, page 2).

Exercise 45 [Fall 2024, Week 11 (“Bonus Week”); *revised*] Consider following RA’s expression(s):

$$\sigma_{year \leq 2010 \vee name = 'Abc'}(\rho_T(takes) + student)$$

1. Write a *language-equivalent* query in SQL standard (cf. Definition 1, page 2).

Exercise 46 [Fall 2024, Week 11 (“Bonus Week”); *revised*] Consider following RA’s expression(s):

$$\sigma_{year \leq 2010 \vee name = 'Abc'}(\rho_J(takes + student))$$

1. Write a *language-equivalent* query in SQL standard (cf. Definition 1, page 2).

Exercise 47 [Fall 2024, Week 11 (“Bonus Week”); *revised*] Consider following RA’s expression(s):

$$\begin{aligned} s &\leftarrow \sigma_{year \geq 2008}(takes) + student \\ t &\leftarrow \rho_{take2008}(s) \\ \Pi_{name, course_id}(t) \end{aligned}$$

1. Write a *language-equivalent* query in SQL standard (cf. Definition 1, page 2).

Exercise 48 [Fall 2024, Week 11 (“Bonus Week”); *revised*] Consider following RA’s expression(s):

$$\begin{aligned} s &\leftarrow student \bowtie_{(student.ID=takes.ID)} takes \\ \Pi_{name, course_id}(s) \end{aligned}$$

1. Write a *language-equivalent* query in SQL standard (cf. Definition 1, page 2).

Exercise 49 [Fall 2024, Week 11 (“Bonus Week”); *revised*] Consider following RA’s expression(s):

$$\begin{aligned} s &\leftarrow \rho_T(instructor) \times \rho_S(instructor) \\ t &\leftarrow \sigma_{T.salary < S.salary \wedge S.dept_name = 'Biology'}(s) \\ \Pi_{T.salary}(t) \end{aligned}$$

1. Write a *language-equivalent* query in SQL standard (cf. Definition 1, page 2).

Exercise 50 [Fall 2024, Week 11 (“Bonus Week”); *revised*] Consider following RA’s expression(s):

$$T.ID \mathcal{G}_{\rho_{sumC}(sum(credits))}(\Pi_{ID, credits}(\rho_T(takes) \bowtie course))$$

1. Write a *language-equivalent* query in SQL standard (cf. Definition 1, page 2).

Exercise 51 [Fall 2024, Week 11 (“Bonus Week”); *revised*] Consider following RA’s expression(s):

$$\Pi_{name, course_id}(\rho_T(\sigma_{year \geq 2008}(takes) \bowtie student))$$

1. Write a *language-equivalent* query in SQL standard (cf. Definition 1, page 2).

4 Translate from TRC

For each TRC's expression in exercises below, if the expression is syntactically correct, then do what asked. Otherwise, explain the error, rewrite the expression correctly and do what asked starting from the corrected expression.

Exercise 52 [Fall 2024, Week 11 (“Bonus Week”); *revised*] Consider following TRC's expression(s):

$$\{t \mid \exists s \in instructor (t[name] = s[name] \wedge s[salary] > 70000)\}$$

1. Write a *language-equivalent* query in SQL standard (cf. Definition 1, page 2).

Exercise 53 [Fall 2024, Week 11 (“Bonus Week”); *revised*] Consider following TRC's expression(s):

$$\{t \mid \exists I \in instructor (t[name] = I[name] \wedge I[salary] > 70000)\}$$

1. Write a *language-equivalent* query in SQL standard (cf. Definition 1, page 2).

Exercise 54 [Fall 2024, Week 11 (“Bonus Week”); *revised*] Consider following TRC's expression(s):

$$\{t \mid \exists s \in instructor (t[name] = s[name] \wedge t[ID] = s[ID] \wedge s[salary] > 70000)\}$$

1. Write a *language-equivalent* query in SQL standard (cf. Definition 1, page 2).

Exercise 55 [Fall 2024, Week 11 (“Bonus Week”); *revised*] Consider following TRC's expression(s):

$$\{t \mid \exists I \in instructor (t[name] = I[name] \wedge t[dept_name] = I[dept_name] \\ \wedge \exists D \in department (t[building] = D[building] \\ \wedge I[dept_name] = D[dept_name]))\}$$

1. Write a *language-equivalent* query in SQL standard (cf. Definition 1, page 2).

Exercise 56 [Fall 2024, Week 11 (“Bonus Week”); *revised*] Consider following TRC's expression(s):

$$\{(I, D) \mid I \in instructor \wedge D \in department\}$$

1. Write a *language-equivalent* query in SQL standard (cf. Definition 1, page 2).

Exercise 57 [Fall 2024, Week 11 (“Bonus Week”); *revised*] Consider following TRC's expression(s):

$$\{t \mid \exists i \in instructor \exists d \in department (t[ID] = i[ID] \wedge t[name] = i[name] \\ \wedge t[i.dept_name] = i[dept_name] \wedge t[salary] = i[salary] \\ \wedge t[d.dept_name] = d[dept_name] \wedge t[building] = d[building] \wedge t[budget] = d[budget])\}$$

1. Write a *language-equivalent* query in SQL standard (cf. Definition 1, page 2).

Exercise 58 [Fall 2024, Week 11 (“Bonus Week”); *revised*] Consider following TRC's expression(s):

$$\{t \mid \exists I \in instructor \exists D \in department (t[ID] = I[ID] \wedge t[name] = I[name] \\ \wedge t[dept_name] = I[dept_name] \wedge t[salary] = I[salary] \\ \wedge t[building] = D[building] \wedge t[budget] = D[budget])\}$$

1. Write a *language-equivalent* query in SQL standard (cf. Definition 1, page 2).

Exercise 59 [Fall 2024, Week 11 (“Bonus Week”); *revised*] Consider following TRC’s expression(s):

$$Q = \{t \mid \exists i \in instructor \exists te \in teaches (t[ID] = i[ID] \wedge i[ID] = te[ID])\} \\ \{t \mid \exists i \in instructor (t[ID] = i[ID] \wedge \dots \wedge \forall q \in Q (i[ID] \neq q[ID]))\}$$

1. Write a *language-equivalent* query in SQL standard (cf. Definition 1, page 2).

Exercise 60 [Fall 2024, Week 11 (“Bonus Week”); *revised*] Consider following TRC’s expression(s):

$$Q_1 = \{t \mid \exists i \in instructor \exists te \in teaches (t[ID] = i[ID] \wedge \dots \wedge t[salary] = i[salary] \\ \wedge t[course_id] = te[course_id] \wedge \dots \wedge t[year] = te[year]) \wedge i[ID] = te[ID])\} \\ Q_2 = \{t \mid \exists q \in Q_1 (t[ID] = q[ID])\} \\ \{t \mid \exists i \in instructor (t[ID] = i[ID] \wedge \forall q \in Q_2 (i[ID] \neq q[ID]))\}$$

1. Write a *language-equivalent* query in SQL standard (cf. Definition 1, page 2).

Exercise 61 [Fall 2024, Week 11 (“Bonus Week”); *revised*] Consider following TRC’s expression(s):

$$\{t \mid \exists i \in takes, \exists s \in student (i[ID] = s[ID] \wedge t[name] = s[name] \\ \wedge t[course_id] = i[course_id] \wedge i[year] \geq 2008)\}$$

1. Write a *language-equivalent* query in SQL standard (cf. Definition 1, page 2).

5 Translate from DRC*

* *Optional section. If all exercises in this section are solved correctly, 10 points extra are granted.*

Exercise 62 [Fall 2024, Week 11 (“Bonus Week”); *revised*] Consider following DRC’s expression(s):

$$\{\langle i, n, d, s, de, b, bg \rangle \mid \langle i, n, d, s \rangle \in instructor \wedge \langle de, b, bg \rangle \in department\}$$

1. Write a *language-equivalent* query in SQL standard (cf. Definition 1, page 2).

Exercise 63 [Fall 2024, Week 11 (“Bonus Week”); *revised*] Consider following DRC’s expression(s):

$$I = \{\langle i, n, d, s \rangle \mid \langle i, n, d, s \rangle \in instructor\} \\ D = \{\langle d, b, bg \rangle \mid \langle d, b, bg \rangle \in department\} \\ \{\langle i, n, d, s, de, b, bg \rangle \mid \langle i, n, d, s \rangle \in I \wedge \langle de, b, bg \rangle \in D\}$$

1. Write a *language-equivalent* query in SQL standard (cf. Definition 1, page 2).

Exercise 64 [Fall 2024, Week 11 (“Bonus Week”); *revised*] Consider following DRC’s expression(s):

$$\{\langle i, n, d, s, b, bg \rangle \mid \langle i, n, d, s \rangle \in instructor \wedge \langle de, b, bg \rangle \in department \\ \wedge d = de\}$$

1. Write a *language-equivalent* query in SQL standard (cf. Definition 1, page 2).

Exercise 65 [Fall 2024, Week 11 (“Bonus Week”); *revised*] Consider following DRC’s expression(s):

$$\{\langle c \rangle \mid \exists a, s, y, b, r, t (\langle c, a, s, y, b, r, t \rangle \in section \\ \wedge (s = 'Fall' \wedge y = 2009) \vee (s = 'Spring' \wedge y = 2010))\}$$

1. Write a *language-equivalent* query in SQL standard (cf. Definition 1, page 2).

Exercise 66 [Fall 2024, Week 11 (“Bonus Week”); *revised*] Consider following DRC’s expression(s):

$$\{\langle n \rangle \mid \exists i, d, s (\langle i, n, d, s \rangle \in instructor \\ \wedge \exists b, g (\langle d, b, g \rangle \in department \wedge b = 'Watson'))\}$$

1. Write a *language-equivalent* query in SQL standard (cf. Definition 1, page 2).

Exercise 67 [Fall 2024, Week 11 (“Bonus Week”); *revised*] Consider following DRC’s expression(s):

$$\{\langle d \rangle \mid \exists i, n, s (\langle i, n, d, s \rangle \in instructor \wedge n = 'Eloise' \vee \neg (s = 80000))\}$$

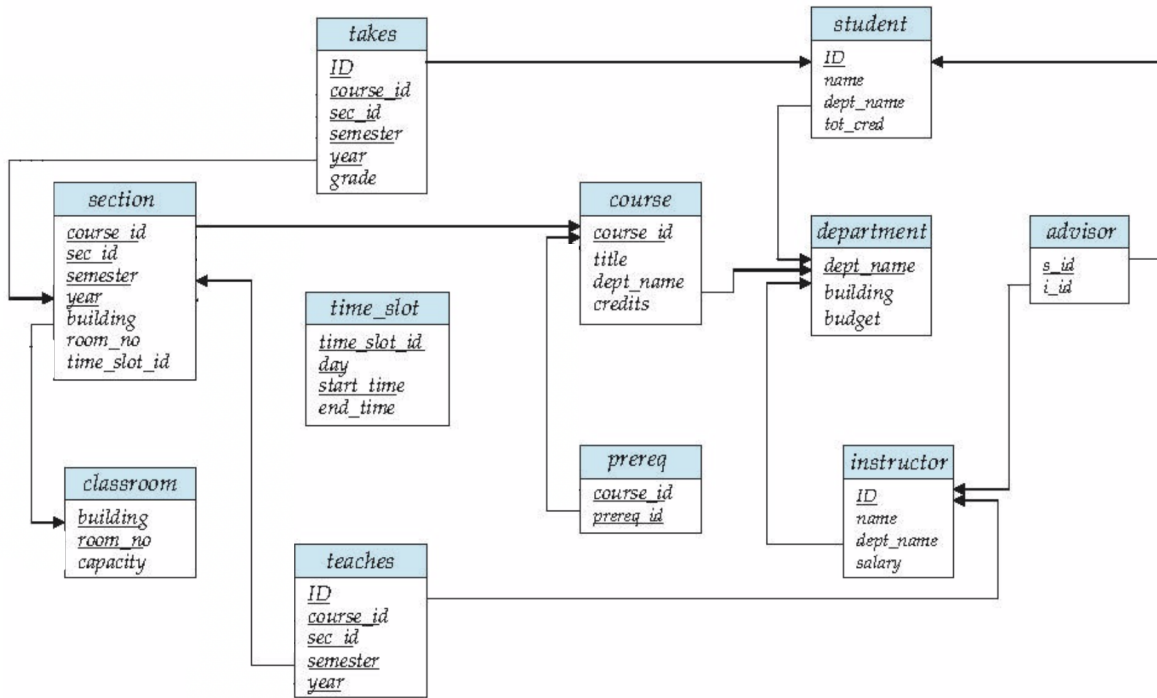
1. Write a *language-equivalent* query in SQL standard (cf. Definition 1, page 2).

Exercise 68 [Fall 2024, Week 11 (“Bonus Week”); *revised*] Consider following DRC’s expression(s):

$$\{\langle n, cid \rangle \mid \exists id, dn, tc, sid, s, y, g (\langle id, n, dn, tc \rangle \in student \wedge \langle id, cid, sid, s, y, g \rangle \in \\ takes \wedge y \geq 2008)\}$$

1. Write a *language-equivalent* query in SQL standard (cf. Definition 1, page 2).

See the next page for the database’s schema to use.

Figure 2: Logical schema of University Database (Silberschatz *et al.*, 2011).